The engineering behind VisionRedact

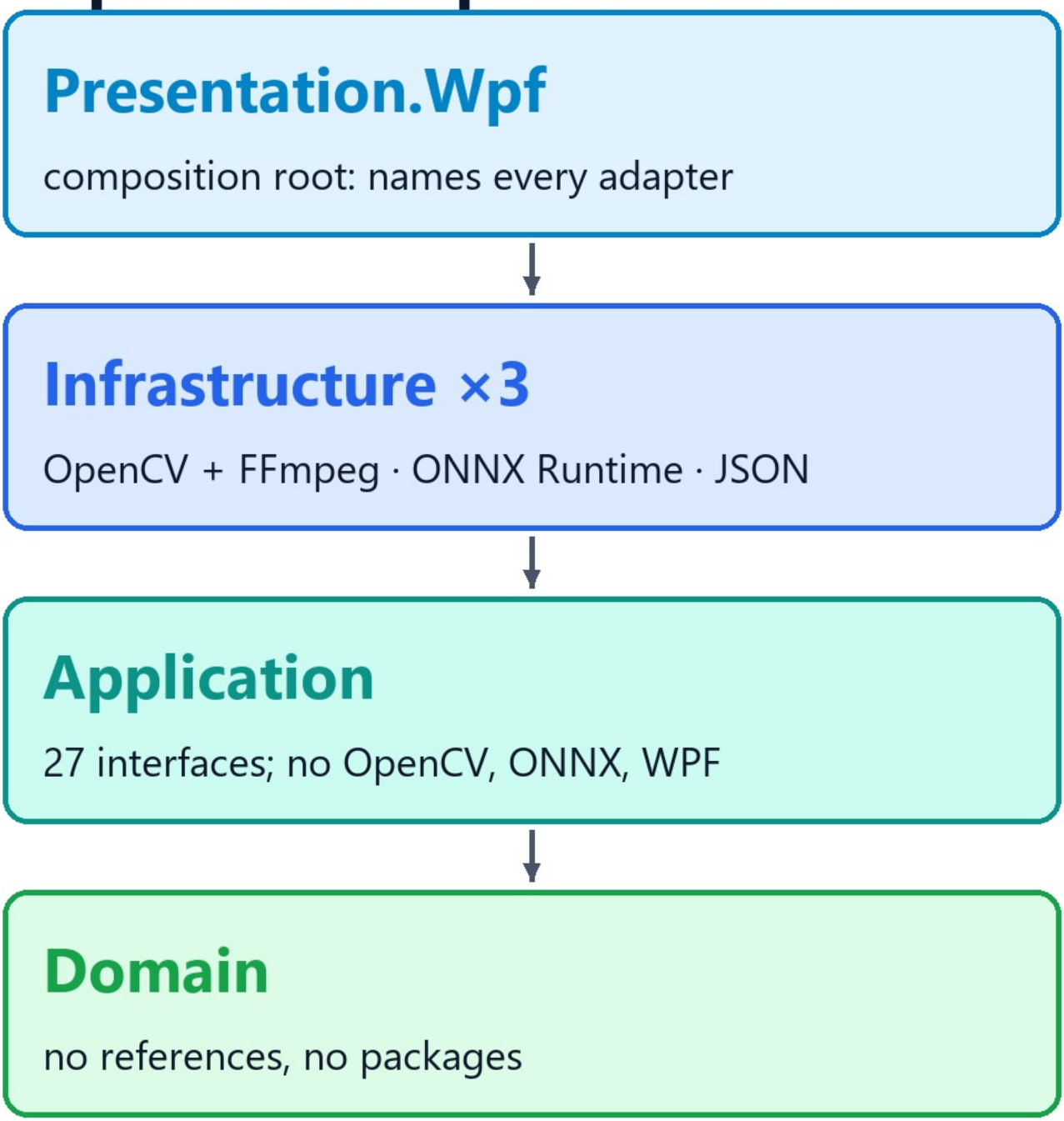
The rule a test enforces, and two measurements that lied. Six projects, dependencies pointing inward, pipelines joined by bounded queues, and the GPU only where it measured a win.



C# · .NET 10 · Clean Architecture · xUnit · CI

Samit Desai · linkedin.com/in/samit-desai-62868420

Dependencies point inward



The rule is checked by a test

1. **Domain**
   may reference only .NET itself

2. **Application**
   no OpenCV, ONNX Runtime, WPF, Prism, Serilog or outer project

3. **Twice**
   compiled references and the project files

4. **CI**
   warnings as errors; 255 of 346 tests (3 timing tests run locally)

Swap a technology, change one place

1. **Port**

   IVideoWriterFactory, owned by Application

2. **Before**

   OpenCV's writer: no audio

3. **After**

   FFmpeg writer wraps it as the fallback

4. **Changed**

   one registration in App.xaml.cs

Threads: who does what

1. **UI thread**

   input, copy the frame, layout, render pass

2. **Playback**

   decode → redact → present, bounded channels

3. **Analysis**

   decode → inference on the GPU

4. **Export**

   decode → redact → encode, plus ffmpeg.exe

CPU vs GPU today, and v2

1. **GPU today**
   face detection: 5× the CPU

2. **Tried, slower**
   blur on the integrated GPU: 13.6 vs 8.3 ms

3. **v2**
   hardware decode, GPU-resident frames, GPU effects

4. **Goal**
   the whole pipeline on the GPU, with a CPU fallback

Measurement that lied #1: the busy machine

Same code, same 4K30 clip.

Runs 1–2

26 % of frames over budget

Run 3

0.7 % of frames over budget

The PC was in use and a stray process held a core. Now: record machine state, mirrored runs, medians.

Measurement that lied #2: cold starts

1. **Tile lookup**
   mean 7.6–11.8 µs, median 2–5 µs

2. **Layout per pass**
   an average that was one first layout (~17.7 ms)

3. **Fix**
   medians and recent windows

Patterns, each with a job

1. **Ports and adapters**
   IVideoDecoder → EmguVideoDecoder

2. **Decorator**
   NmsDecoder wraps a model's decoder and adds NMS

3. **Strategy**
   IRedactionEffect ×3, IModelFormatProfile, sampling

4. **Composite**
   CompositeDetector in DetectorHost

5. **Factory with CPU fallback**
   OnnxDetectorFactory: DirectML; CPU if it can't load the model

6. **Producer–consumer**
   bounded channels: playback, analysis, export

Trade-offs, and the series

- Six projects and 27 interfaces: some indirection.
- Integration tests need GPUs, models and video: run locally.
- Decode, effects and encode stay on the CPU, for now.

The series

Thanks for following along.

Where it's going: the whole pipeline on the GPU.